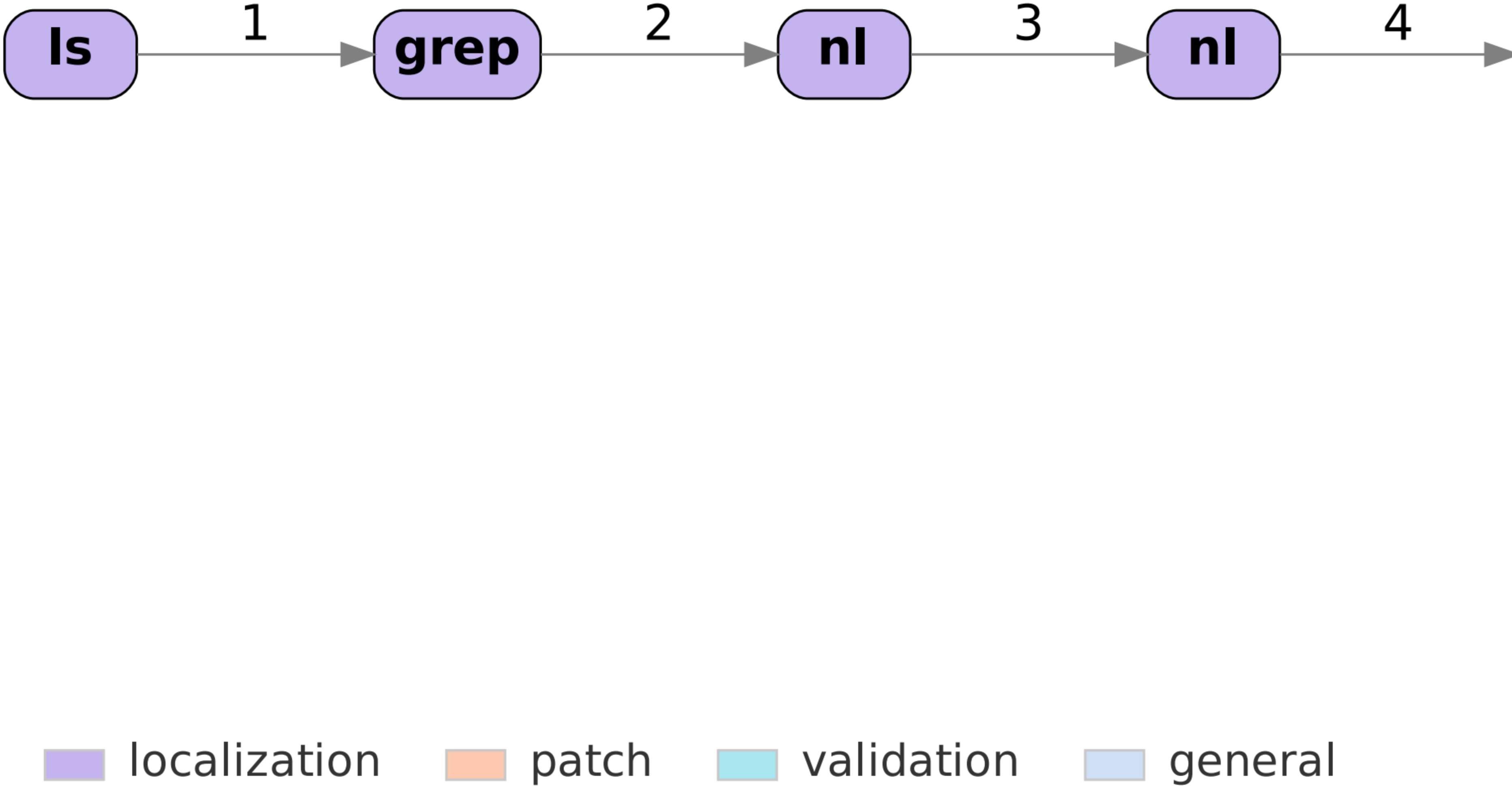
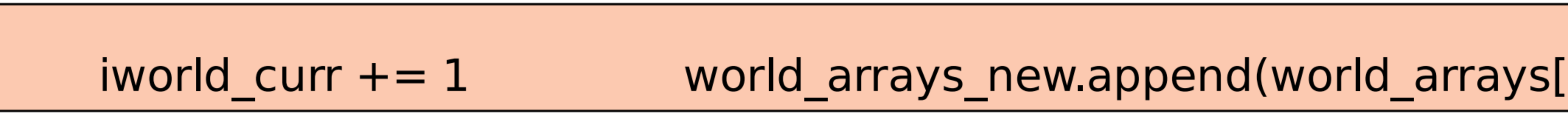


```
.../sliced_wcs.py')s=p.read_text()old='''        world_arrays = tuple(map(np.asa
```

```
nyarray, world_arrays))        world_arrays_new = []        iworld_curr = -1        for
```

```
iworld in range(self._wcs.world_n_dim):        if iworld in self._world_keep:
```



```
iworld_curr += 1
world_arrays_new.append(world_arrays[
```

```
iworld_curr])          else:               world_arrays_new.append(1.)'''new='''
```

```
world_arrays = tuple(map(np.asanyarray, world_arrays))        world_arrays_n
```

```
ew = []        iworld_curr = -1        # prepare dropped world values for world di
```

ms removed by slicing	dropped_indices = [i for i in range(self._wcs

```
.world_n_dim) if i not in self._world_keep]        try:            dropped_info =
```

python

```
self.dropped_world_dimensions         dropped_values = dropped_info.get('val
```

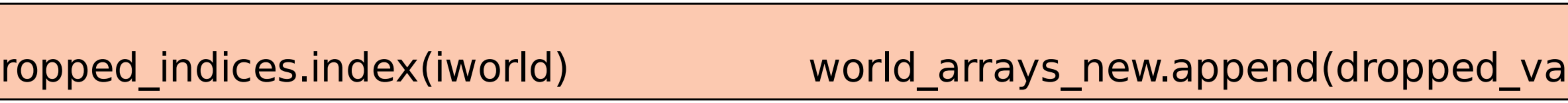
```
ue', []) if isinstance(dropped_info, dict) else []        except exception:          d
```

```
ropped_values = []        for iworld in range(self._wcs.world_n_dim):            if
```

```
iworld in self._world_keep:            iworld_curr += 1            world_a
```

```
rrays_new.append(world_arrays[iworld_curr])          else:                  if d
```

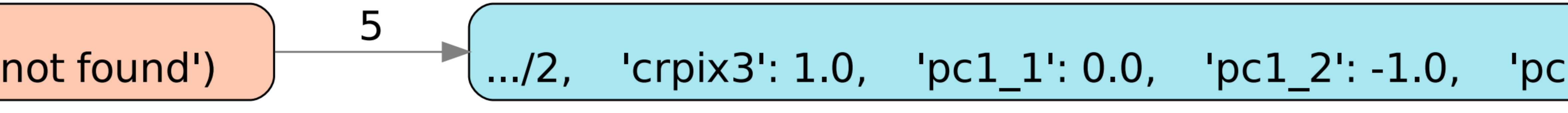
```
ropped_values:            # find index in dropped_indices            idx = d
```



```
ropped_indices.index(iworld)                world_arrays_new.append(dropped_va
```

```
lues[idx])            else:                world_arrays_new.append(1.)'''if old in s:
```

```
s=s.replace(old,new)    p.write_text(s)    print('patched')else:    print('pattern
```


not found')
5
.../2,   'crpix3': 1.0,   'pc1_1': 0.0,   'pc1_2': -1.0,   'pc

1_3': 0.0, 'pc2_1': 1.0, 'pc2_2': 0.0, 'pc2_3': -1.0, 'cdelt1': 5, 'cdelt2': 5,

```
'cdelt3': 0.055,    'cunit1': 'arcsec',    'cunit2': 'arcsec',    'cunit3': 'angstrom',
```

```
'ctype1': 'hpln-tan',    'ctype2': 'hplt-tan',    'ctype3': 'wave',    'crval1': 0.0,    'c
```

python

```
rval2': 0.0,    'crval3': 1.05,}fits_wcs = astropy.wcs.wcs(header=wcs_header)pt
```

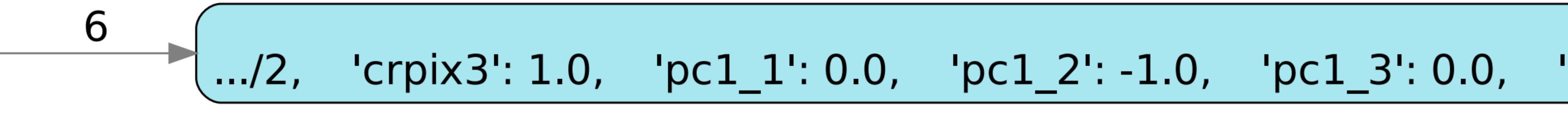
```
= skycoord(tx=0*u.arcsec, ty=0*u.arcsec, frame=wcs_to_celestial_fra
```

```
me(fits_wcs))full_px = fits_wcs.world_to_pixel(pt, 1.05*u.angstrom)ll_sl
```

```
iced_wcs = slicedlowlevelwcs(fits_wcs, 0)hl_sliced_wcs = highlevelwcswra
```

```
pper(ll_sliced_wcs)sliced_px = hl_sliced_wcs.world_to_pixel(pt)print('full
```

```
world_to_pixel:', full_px)print('sliced world_to_pixel:', sliced_px)
```


6
.../2, 'crpix3': 1.0, 'pc1_1': 0.0, 'pc1_2': -1.0, 'pc1_3': 0.0, '

```
pc2_1': 1.0,    'pc2_2': 0.0,    'pc2_3': -1.0,    'cdelt1': 5,    'cdelt2': 5,    'cdelt3':
```

```
0.055,    'cunit1': 'arcsec',    'cunit2': 'arcsec',    'cunit3': 'angstrom',    'ctype1': '
```

```
hpln-tan',    'ctype2': 'hplt-tan',    'ctype3': 'wave',    'crval1': 0.0,    'crval2': 0.0,
```

```
'crval3': 1.05,}fits_wcs = wcs(header=wcs_header)# choose the pixel
```

```
coords from pr examplepx,py,pz = 49.5, 12., 1.0# get world values from full
```

```
wcsworld_vals = fits_wcs.pixel_to_world_values(px, py, pz)print("p
```

```
ixel_to_world_values (full):", world_vals, type(world_vals))# now round-trip with
```

```
full wcspix_back = fits_wcs.world_to_pixel_values(*world_vals)print(
```

```
"world_to_pixel_values (full):", pix_back)# create sliced wcs (slice first pixel di
```

python

```python
mension)ll_sliced = slicedlowlevelwcs(fits_wcs, 0)# for sliced wcs, pass the
```

remaining world dims (should be 2 dims: spatial)# determine what world_to_p ixel_values expects by inspecting world_n_dimprint("full world_n_dim:",

```
fits_wcs.world_n_dim, "sliced world_n_dim:", ll_sliced.world_n_dim)# extract
```

the kept world values from world_vals according to sliced world_n_dim#

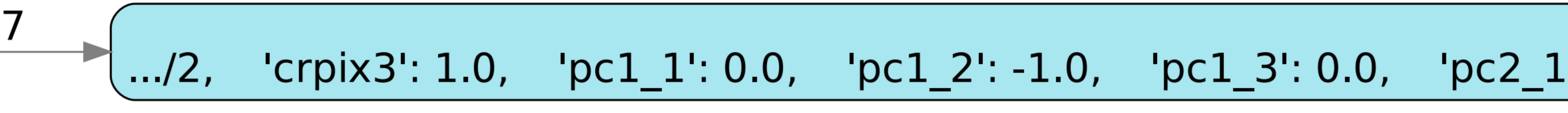
```
world_vals may be a tuple; make it list-likeif isinstance(world_vals, tuple):
```

```
world_vals_full = list(world_vals)else:    world_vals_full = [world_vals]# for sl
```

iced, pass only kept world dims via ll_sliced.world_axis_names mapping by i ndex# we'll try calling with the first two components (spatial)sliced_world_args

```
= world_vals_full[:ll_sliced.world_n_dim]sliced_pix = ll_sliced.world_to_pixel_va
```

```
lues(*sliced_world_args)print("sliced world_to_pixel_values:", sliced_pix)
```


7
.../2, 'crpix3': 1.0, 'pc1_1': 0.0, 'pc1_2': -1.0, 'pc1_3': 0.0, 'pc2_1

```
': 1.0,    'pc2_2': 0.0,    'pc2_3': -1.0,    'cdelt1': 5,    'cdelt2': 5,    'cdelt3': 0.055,
```

```
'cunit1': 'arcsec',    'cunit2': 'arcsec',    'cunit3': 'angstrom',    'ctype1': 'hpln-ta
```

```
n',    'ctype2': 'hplt-tan',    'ctype3': 'wave',    'crval1': 0.0,    'crval2': 0.0,    'crva
```

```
l3': 1.05,}fits_wcs = wcs(header=wcs_header)ll_sliced = slicedlow
```

```
levelwcs(fits_wcs, 0)print("slices_array:", ll_sliced._slices_array)print("slices_p
```

```
ixel:", ll_sliced._slices_pixel)print("pixel_keep:", ll_sliced._pixel_keep)print(
```

```
"world_keep:", ll_sliced._world_keep)print("full world_n_dim:", fits_wcs
```

```
.world_n_dim, "sliced world_n_dim:", ll_sliced.world_n_dim)print("drop
```

```
ped_world_dimensions:", ll_sliced.dropped_world_dimensions)# simulate in
```

puts used earlier: world_vals from full pixel_to_world_values of px,py,pzpx,py,

```
pz = 49.5,12.,1.0world_vals = fits_wcs.pixel_to_world_values(px,py,pz)print(
```

```
"world_vals from full:", world_vals)# reconstruct world_arrays_new as in patc
```

python

```python
hed codeworld_arrays = tuple(map(np.asanyarray, world_vals))dropped_ind
```

```
ices = [i for i in range(fits_wcs.world_n_dim) if i not in ll_sliced._world_keep]d
```

```
ropped_info = ll_sliced.dropped_world_dimensionsdropped_values = dropped_i
```

```
nfo.get('value', []) if isinstance(dropped_info, dict) else []print("dropped_indices
```

```
:", dropped_indices)print("dropped_values:", dropped_values)world_a
```

```
rrays_new=[]iworld_curr=-1for iworld in range(fits_wcs.world_n_dim):    if
```

```
iworld in ll_sliced._world_keep:        iworld_curr+=1        world_arrays_new.ap
```

```
pend(world_arrays[iworld_curr])    else:        if dropped_values:            idx = d
```

```
ropped_indices.index(iworld)          world_arrays_new.append(dropped_values[
```

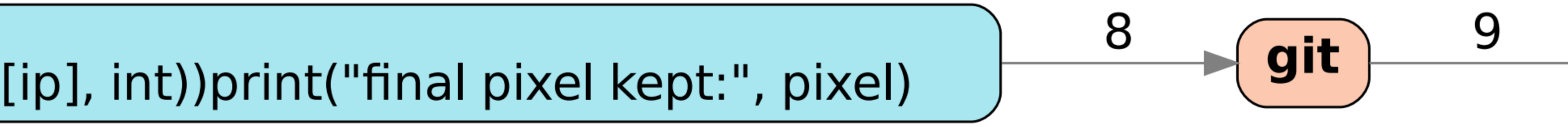
```
idx])      else:          world_arrays_new.append(1.)print("world_arrays_new
```

```
used:", world_arrays_new)# compute pixel arrays from full wcspixarr =
```

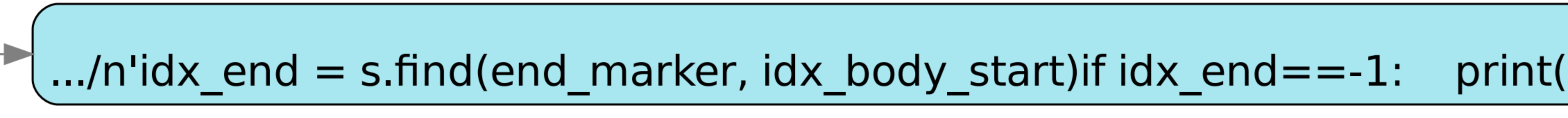
```
fits_wcs.world_to_pixel_values(*world_arrays_new)print("pixel arrays from full
```

```
world_to_pixel_values:", pixarr)# show final sliced pixel tuplepixel = tuple(pixa
```

```
rr[ip] for ip in range(fits_wcs.pixel_n_dim) if not isinstance(ll_sliced._slices_pixel
```



```
[ip], int))print("final pixel kept:", pixel)
```



```
.../n'idx_end = s.find(end_marker, idx_body_start)if idx_end==-1:    print(
```

```
'end marker not found'); raise systemexit(1)old_body = s[idx_body_start:
```

```
idx_end]new_body = """        # convert inputs to arrays        world_arrays = tup
```

```
le(map(np.asanyarray, world_arrays))     # build initial full world array list p
```

```
lacing provided kept world arrays      world_arrays_new = []      iworld_curr =
```

```
-1        dropped_indices = [i for i in range(self._wcs.world_n_dim) if i not in
```

```
self._world_keep]       try:          dropped_info = self.dropped_world_di
```

```
mensions          dropped_values = dropped_info.get('value', []) if isinstance(d
```

```
ropped_info, dict) else []      except exception:          dropped_values = []
```

```
for iworld in range(self._wcs.world_n_dim):          if iworld in self._world_keep:
```

```
iworld_curr += 1
world_arrays_new.append(world_arrays[
```

```
iworld_curr])         else:             # use any available default dropped value as
```

```
initial guess          if dropped_values:          idx = dropped_indices.ind
```

```
ex(iworld)          world_arrays_new.append(dropped_values[idx])
```

```
else:          world_arrays_new.append(1.)     # broadcast to matching sh
```

```
apes        world_arrays_new = np.broadcast_arrays(*world_arrays_new)        #
```

first pass: map to pixel space	pixel_arrays = list(self._wcs.world_to_p

```
ixel_values(*world_arrays_new))      # enforce any integer pixel slices to their
```

fixed pixel indices (they are in _slices_pixel)   for ipixel in range(self._wcs.

```
pixel_n_dim):          if isinstance(self._slices_pixel[ipixel], numbers.integral):
```

```
# ensure pixel_arrays uses the sliced pixel index          pixel_arrays[ip
```

python

```
ixel] = self._slices_pixel[ipixel]      # convert back to world to get improved
```

```
estimates for dropped world axes        try:            world_from_pixels = self._wcs.
```

pixel_to_world_values(*pixel_arrays)   except exception:

world_from_pixels = none   if world_from_pixels is not none: # update dropped world entries with the values computed from the enforced pixel ind

```
ices        if isinstance(world_from_pixels, tuple):        world_from_pixels_l
```

```
ist = list(world_from_pixels)          else:             world_from_pixels_list =
```

[world_from_pixels]

```
for iworld in range(self._wcs.world_n_dim):
```

```
if iworld not in self._world_keep:                    # replace with computed world val
```

```
ue                world_arrays_new[iworld] = world_from_pixels_list[iworld]       #
```

final broadcast and compute pixel positions from these refined world values

```
world_arrays_new = np.broadcast_arrays(*world_arrays_new)        pixel_a
```

```
rrays = list(self._wcs.world_to_pixel_values(*world_arrays_new))        # su
```

```
btract slice starts for non-integer sliced pixels        for ipixel in range(self._wcs.p
```

```
ixel_n_dim):          if isinstance(self._slices_pixel[ipixel], slice) and self._slices_p
```

```
ixel[ipixel].start is not none:            pixel_arrays[ipixel] -= self._slices_pixel[ip
```

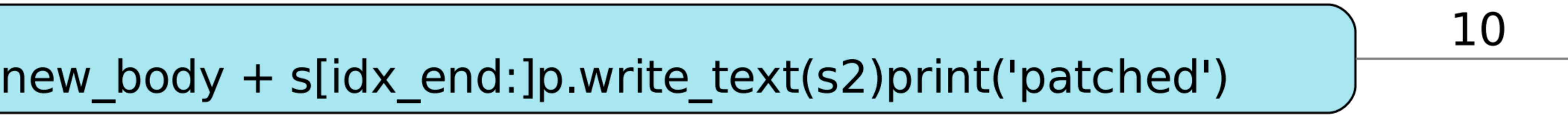
```
ixel].start        # detect the case of a length 0 array        if isinstance(pixel_a
```

```
rrays, np.ndarray) and not pixel_arrays.shape:        return pixel_arrays      #
```

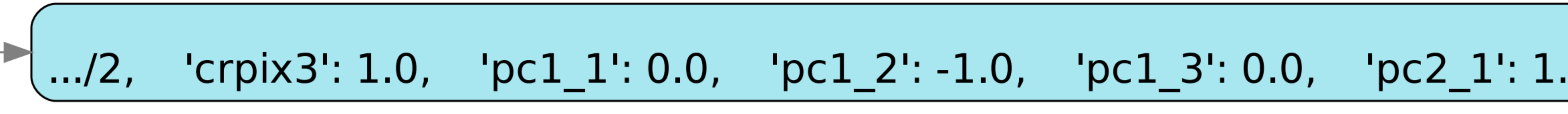
```
return only the kept pixel dimensions      pixel = tuple(pixel_arrays[ip] for ip in
```

```
self._pixel_keep)        if self.pixel_n_dim == 1 and self._wcs.pixel_n_dim > 1:
```

```
pixel = pixel[0]        return pixel"""# replaces2 = s[:idx_body_start] +
```



```
new_body + s[idx_end:]p.write_text(s2)print('patched')
```


.../2, 'crpix3': 1.0, 'pc1_1': 0.0, 'pc1_2': -1.0, 'pc1_3': 0.0, 'pc2_1': 1.

```
0,    'pc2_2': 0.0,    'pc2_3': -1.0,    'cdelt1': 5,    'cdelt2': 5,    'cdelt3': 0.055,    'c
```

```
unit1': 'arcsec',    'cunit2': 'arcsec',    'cunit3': 'angstrom',    'ctype1': 'hpln-tan',
```

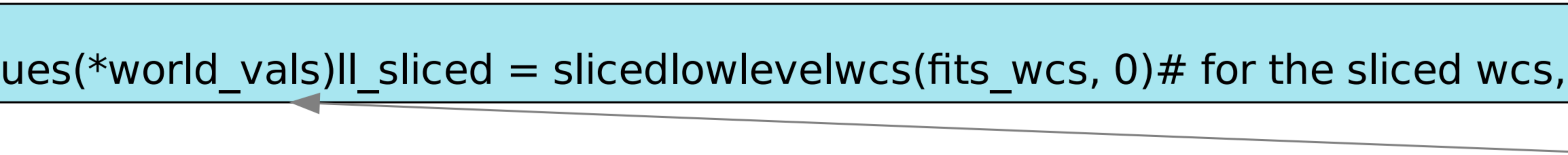
```
'ctype2': 'hplt-tan',    'ctype3': 'wave',    'crval1': 0.0,    'crval2': 0.0,    'crval3':
```

```
1.05,}fits_wcs = wcs(header=wcs_header)# use known pixel coordinates to
```

```
get world valuespx, py, pz = 49.5, 12., 1.0world_vals = fits_wcs.
```

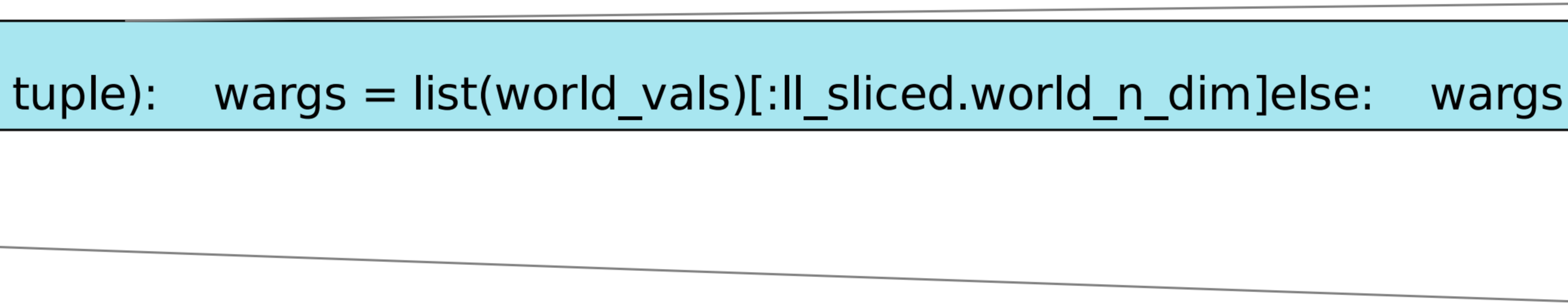
python

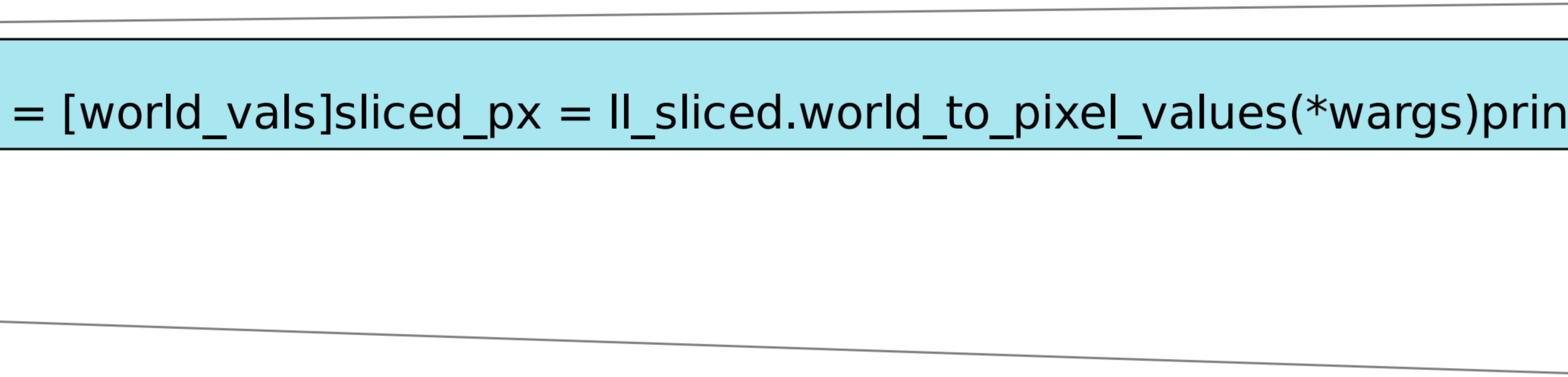
```
pixel_to_world_values(px, py, pz)full_px = fits_wcs.world_to_pixel_val
```



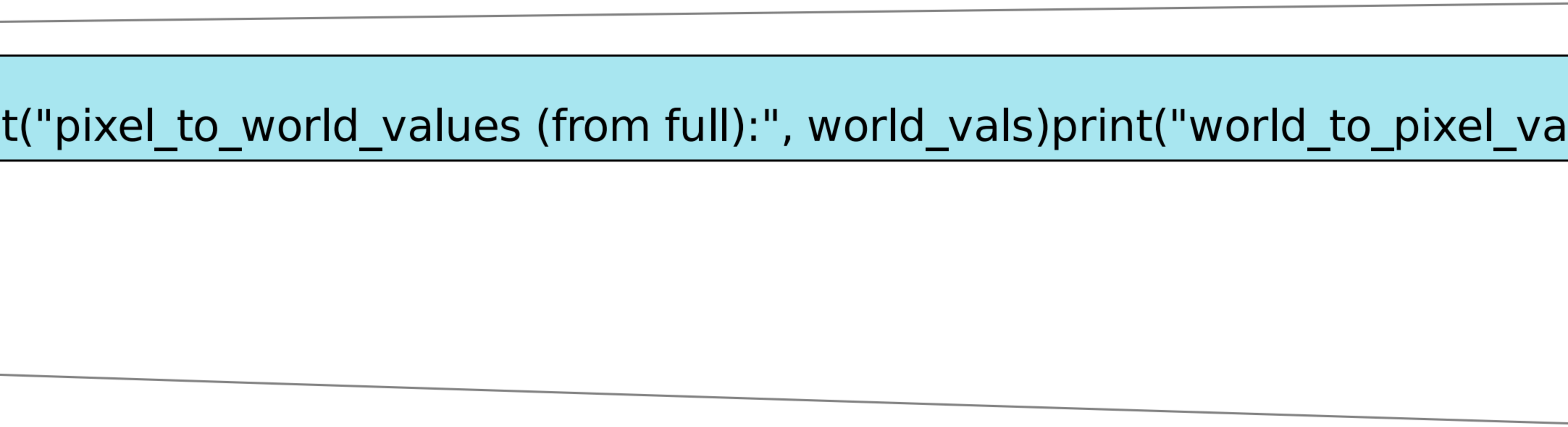
```
ues(*world_vals)ll_sliced = slicedlowlevelwcs(fits_wcs, 0)# for the sliced wcs,
```

pass only the kept world dims (spatial components)if isinstance(world_vals,

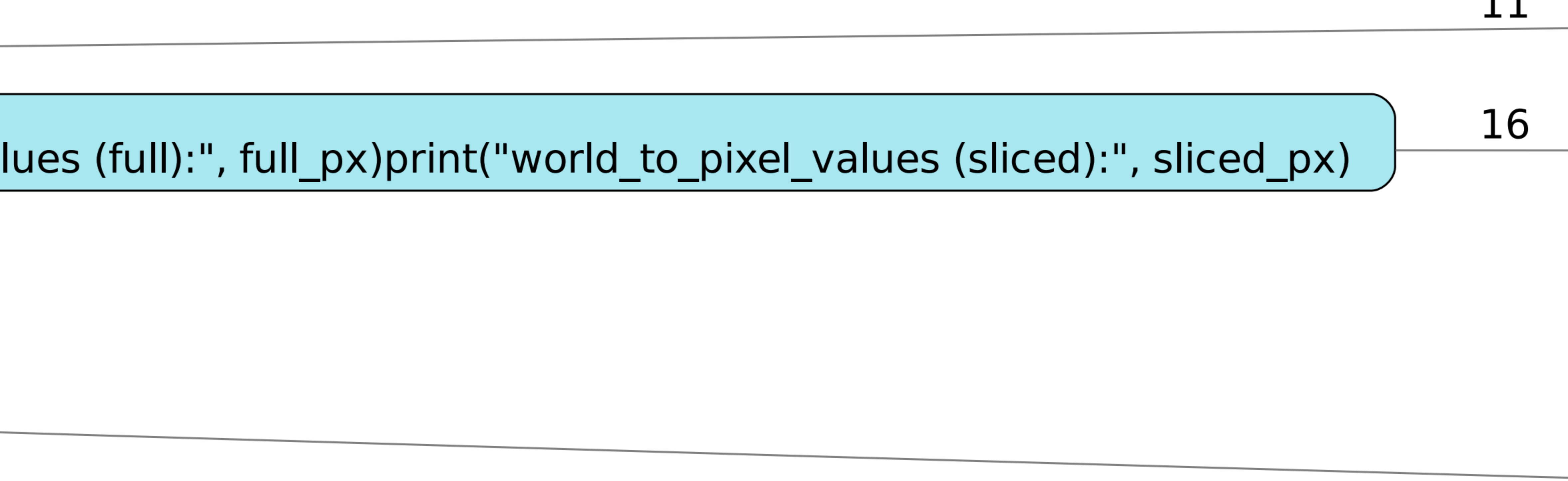

tuple):   wargs = list(world_vals)[:ll_sliced.world_n_dim]else: wargs



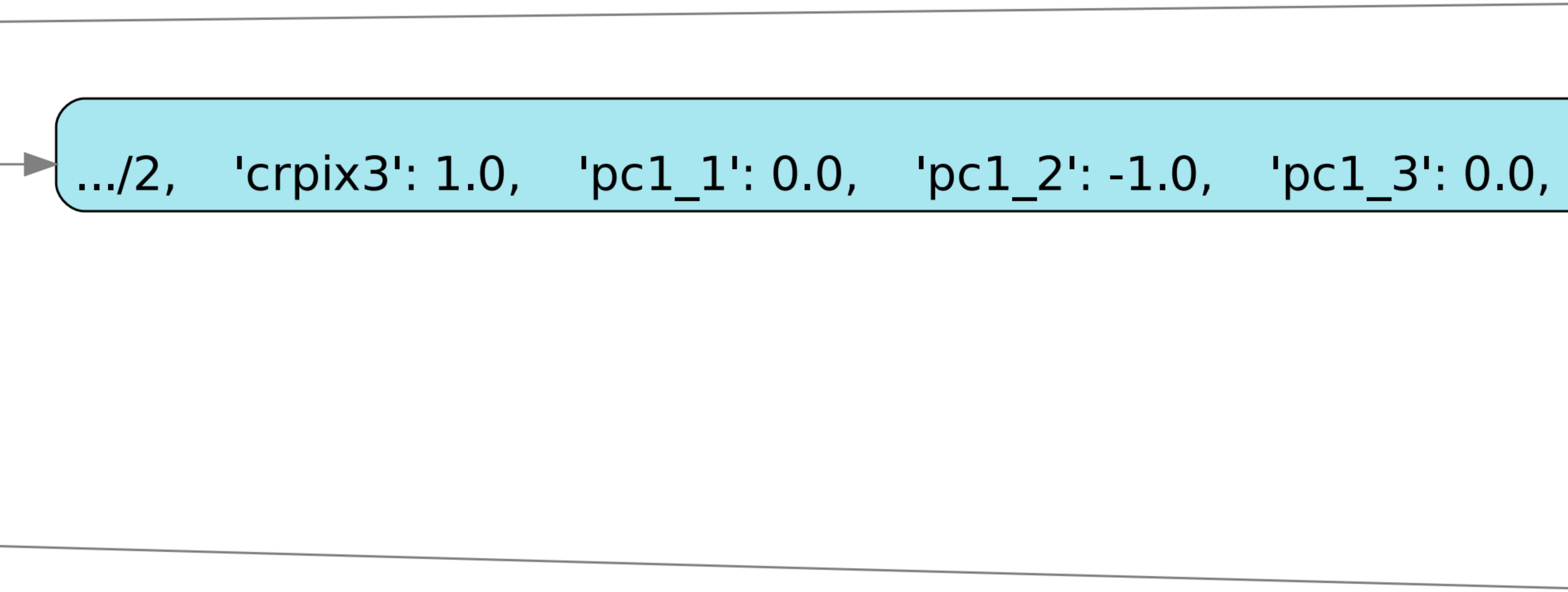
```
= [world_vals]sliced_px = ll_sliced.world_to_pixel_values(*wargs)prin
```

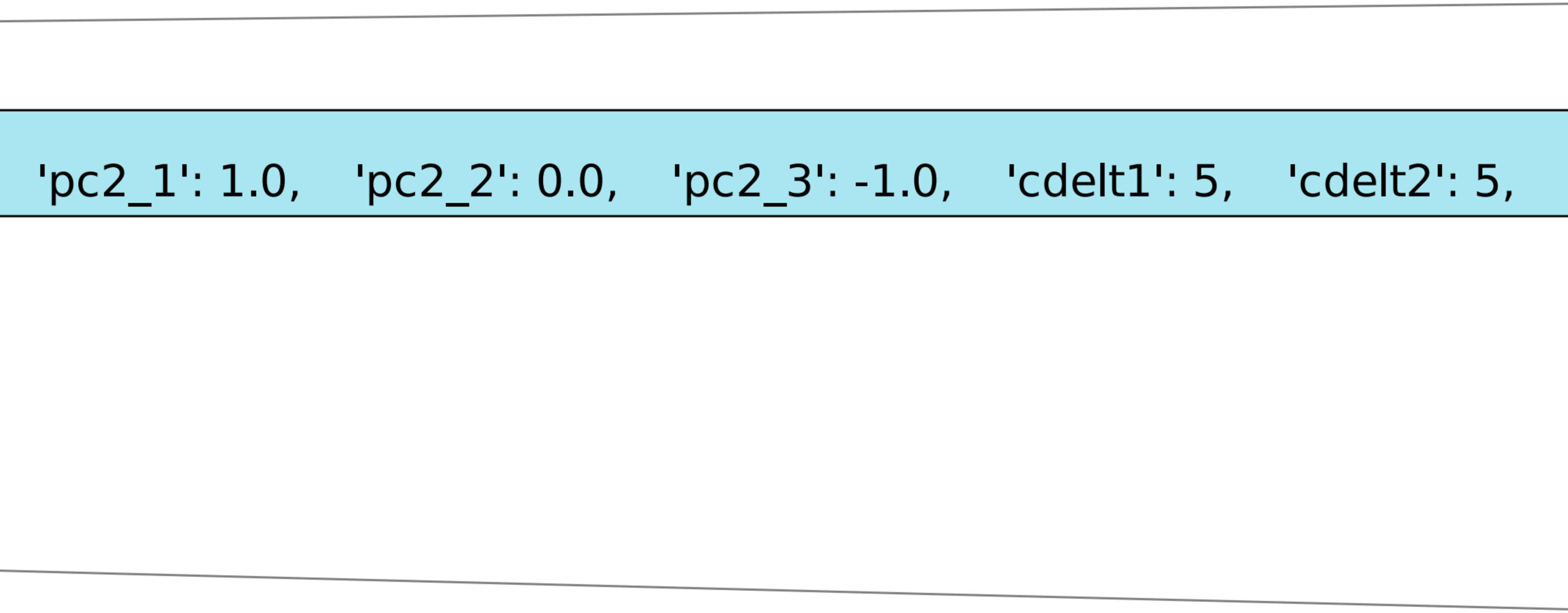


```
t("pixel_to_world_values (from full):", world_vals)print("world_to_pixel_va
```



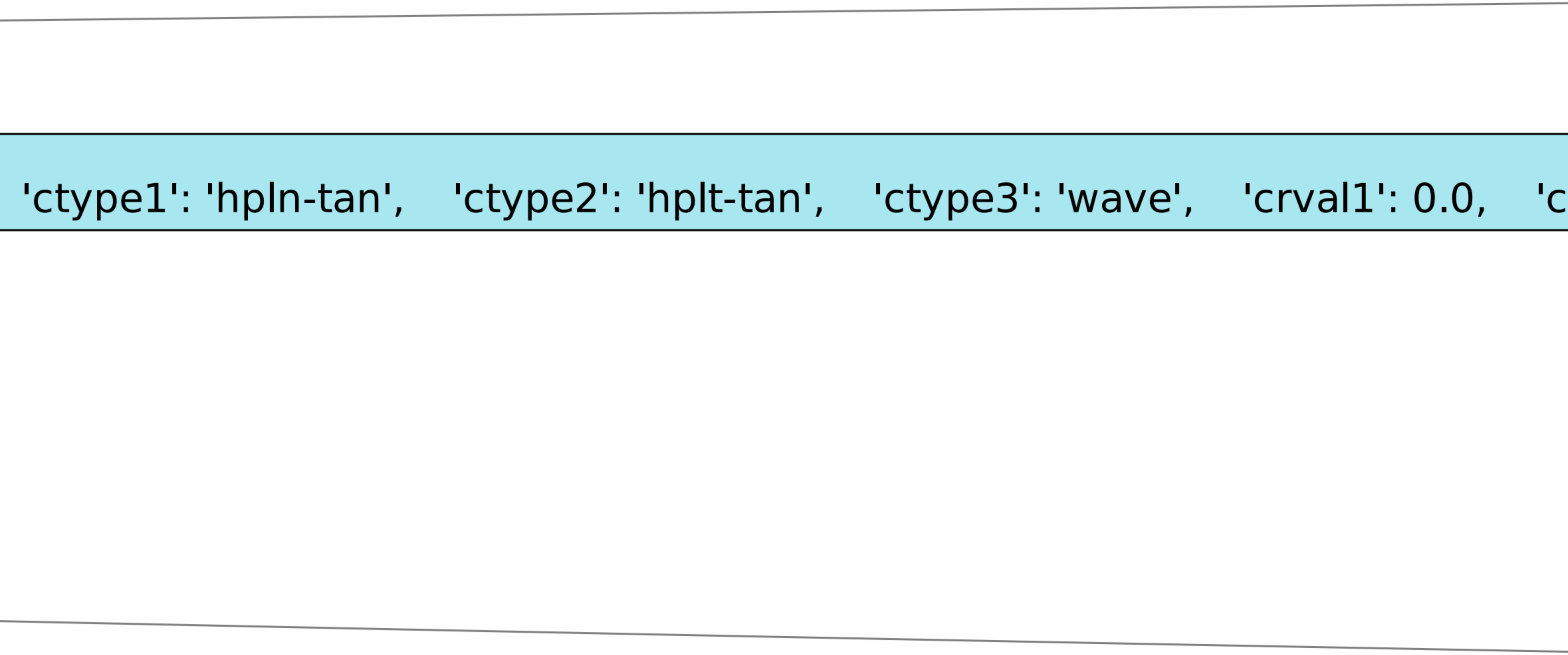
```
lues (full):", full_px)print("world_to_pixel_values (sliced):", sliced_px)
```

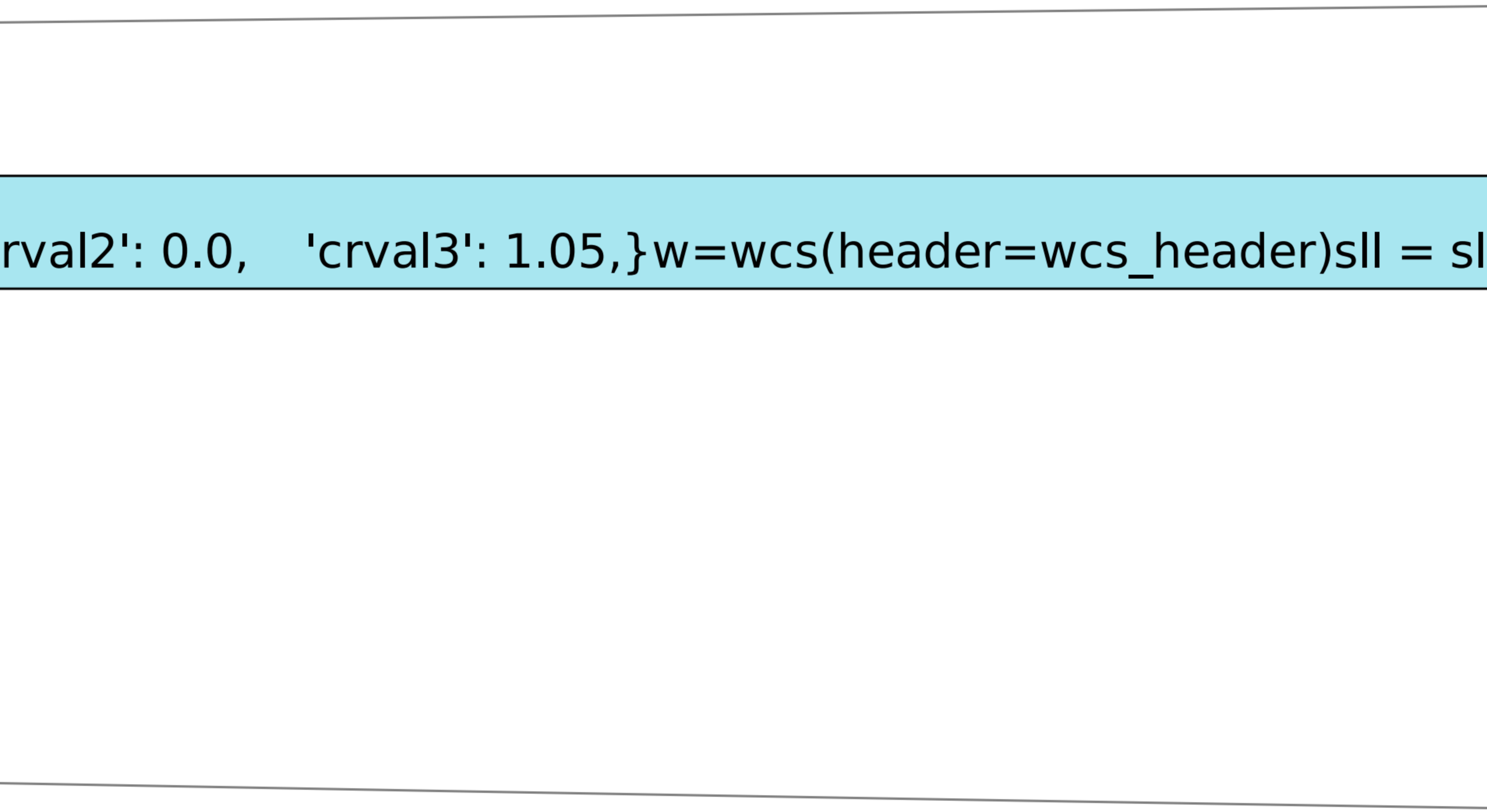

.../2,   'crpix3': 1.0,   'pc1_1': 0.0,   'pc1_2': -1.0,   'pc1_3': 0.0,



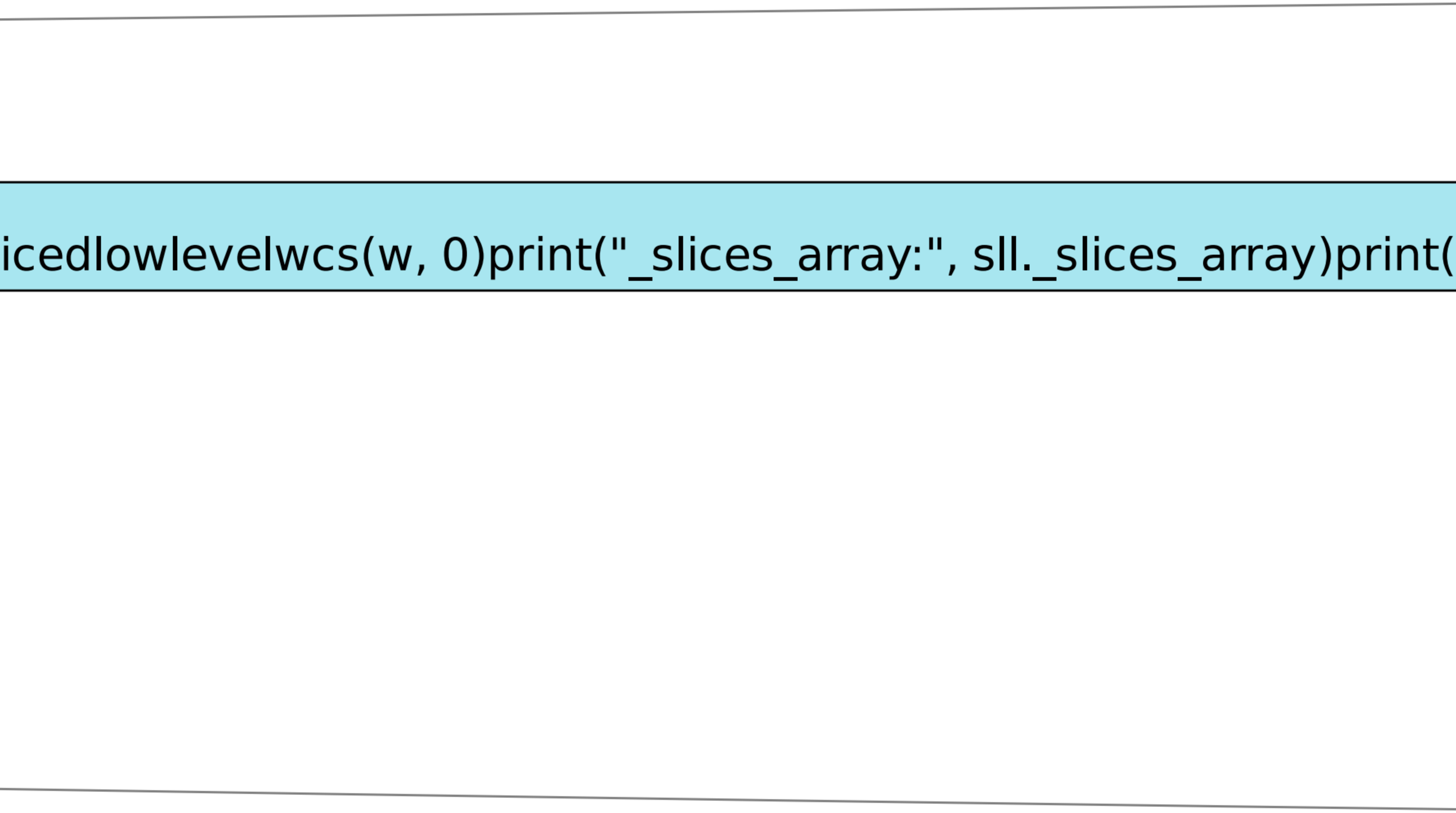
```
'pc2_1': 1.0,    'pc2_2': 0.0,    'pc2_3': -1.0,    'cdelt1': 5,    'cdelt2': 5,
```

```
'cdelt3': 0.055,    'cunit1': 'arcsec',    'cunit2': 'arcsec',    'cunit3': 'angstrom',
```

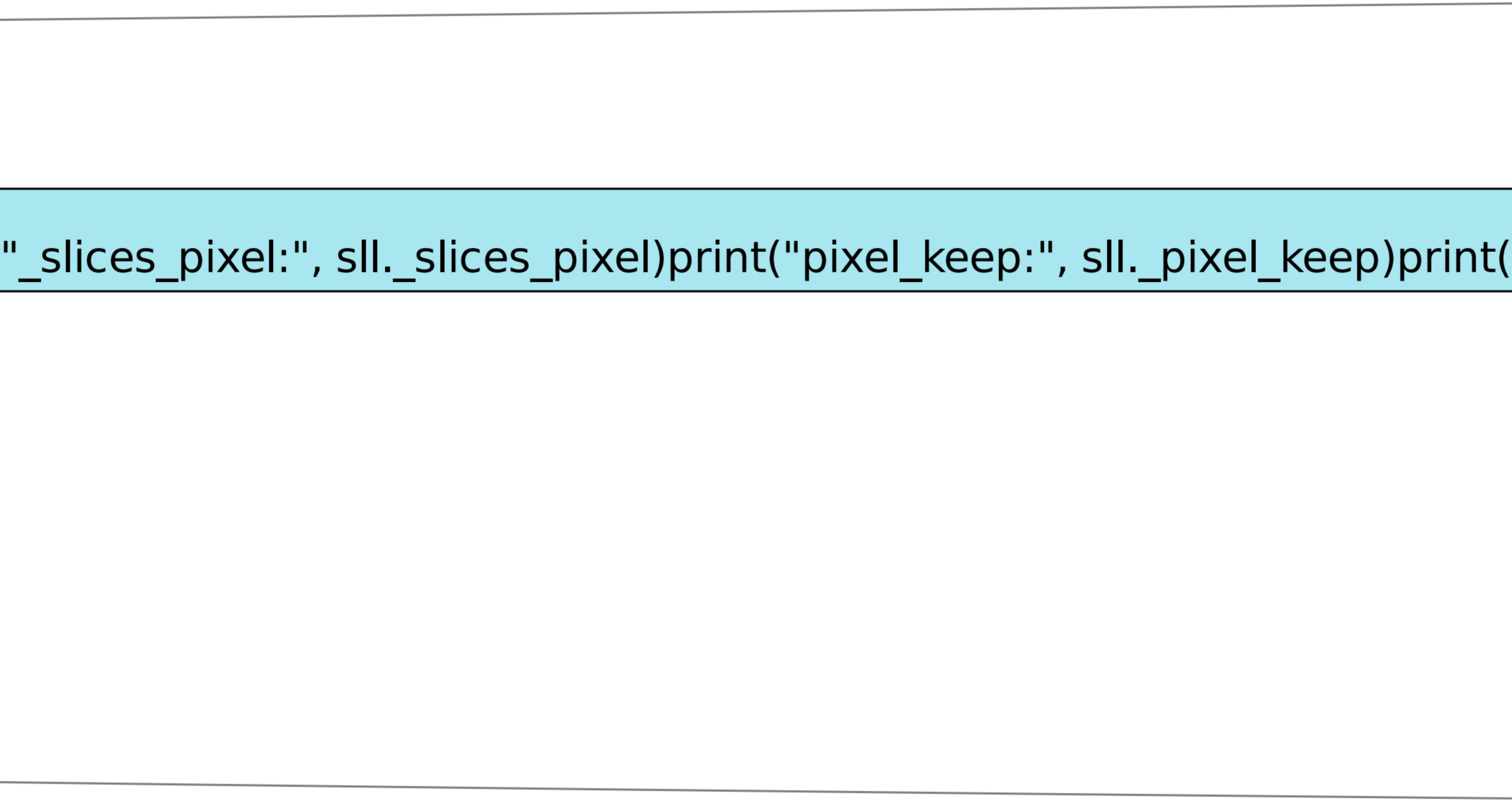




```
rval2': 0.0,    'crval3': 1.05,}w=wcs(header=wcs_header)sll = sl
```

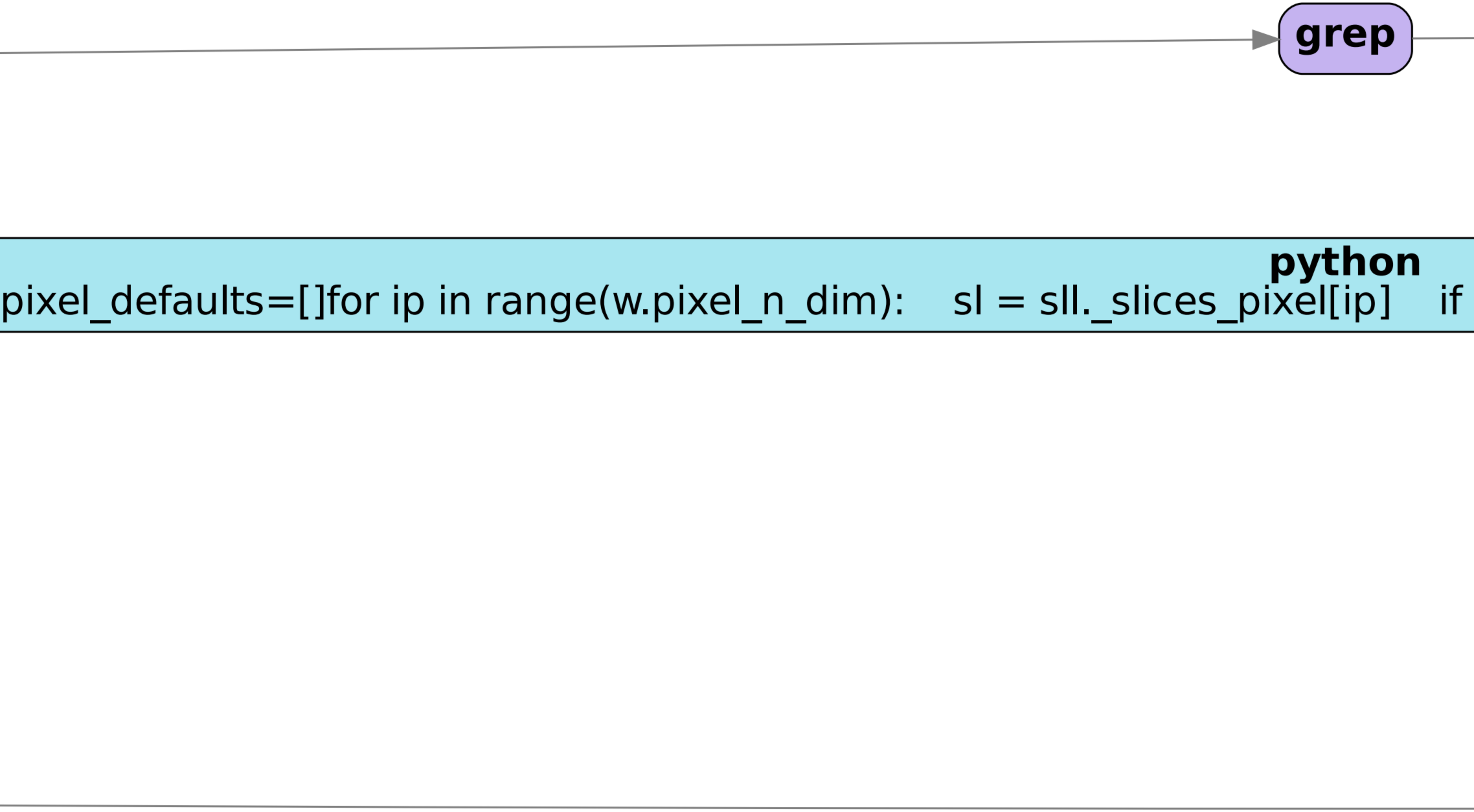


```
icedlowlevelwcs(w, 0)print("_slices_array:", sll._slices_array)print(
```



```
"_slices_pixel:", sll._slices_pixel)print("pixel_keep:", sll._pixel_keep)print(
```

```
"world_keep:", sll._world_keep)# build pixel_defaults as in implementation
```



```
pixel_defaults=[]for ip in range(w.pixel_n_dim):    sl = sll._slices_pixel[ip]    if
```

```
isinstance(sl, numbers.integral):       pixel_defaults.append(sl)    elif isinstan
```

```
ce(sl, slice) and sl.start is not none:       pixel_defaults.append(sl.start)    else:
```

```
pixel_defaults.append(0)print("pixel_defaults:", pixel_defaults)wf =
```

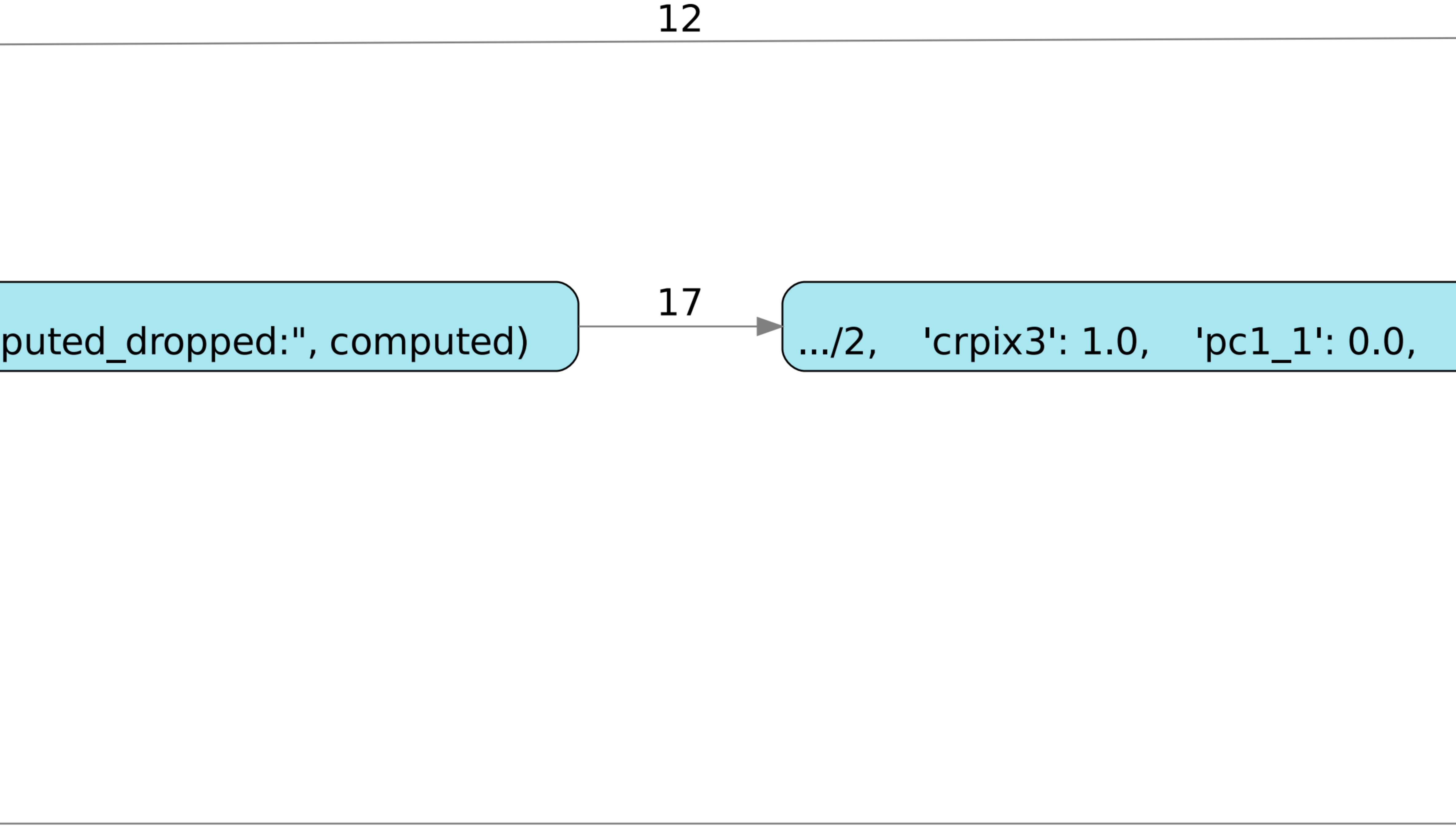
```
w.pixel_to_world_values(*pixel_defaults)print("underlying pixel_to_world_val
```

```
ues for defaults:", wf)# compute dropped mappingdropped_indices = [i for i i
```

```
n range(w.world_n_dim) if i not in sll._world_keep]if isinstance(wf, tuple):
```

```
wf_list = list(wf)else:    wf_list = [wf]computed = {idx: wf_list[idx] for idx
```

```
in dropped_indices}print("dropped_indices:", dropped_indices)print("com
```


12
puted_dropped:", computed)
17
.../2,   'crpix3': 1.0,   'pc1_1': 0.0,

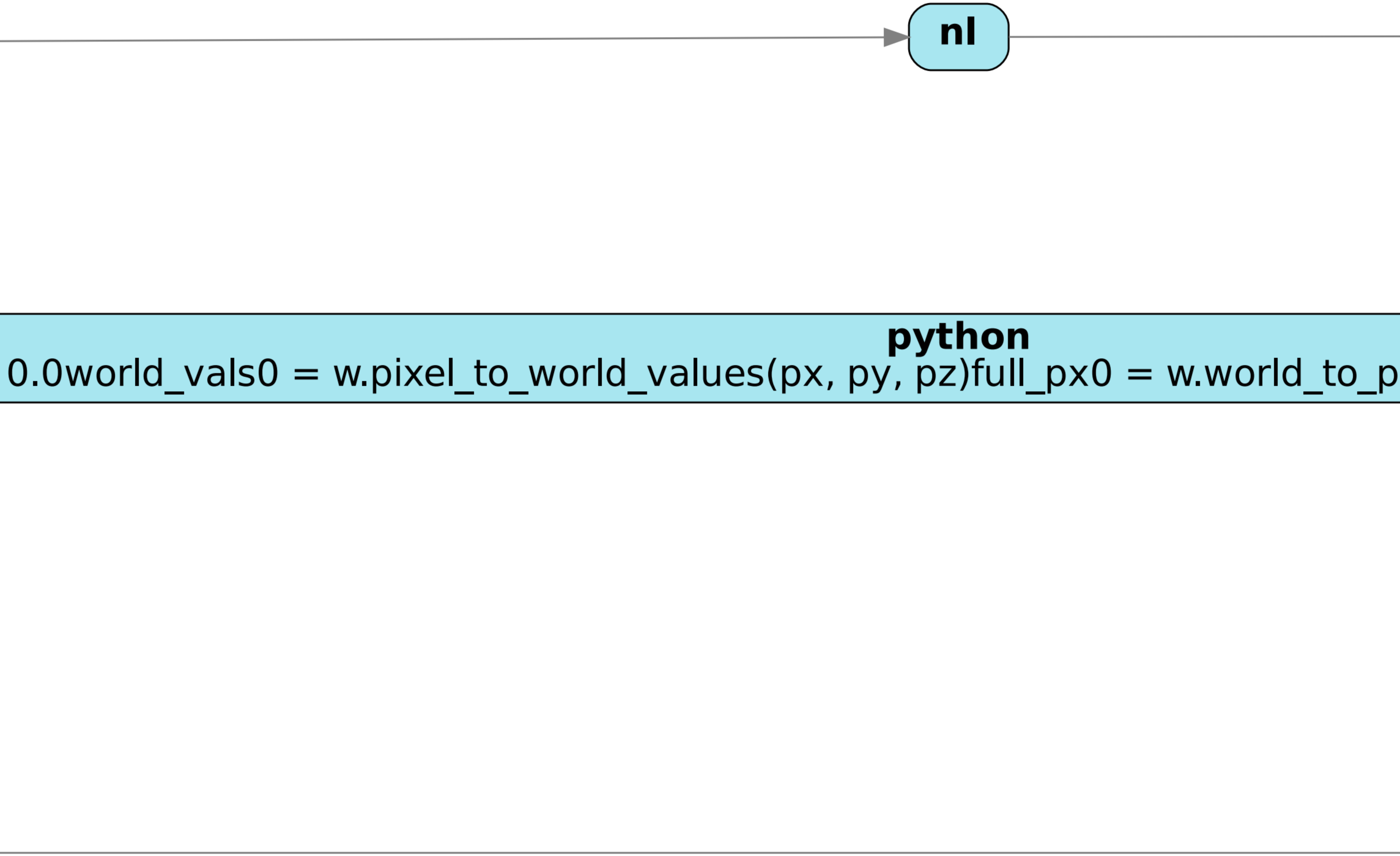
'pc1_2': -1.0, 'pc1_3': 0.0, 'pc2_1': 1.0, 'pc2_2': 0.0, 'pc2_3': -1.0, 'cd

```
elt1': 5,    'cdelt2': 5,    'cdelt3': 0.055,    'cunit1': 'arcsec',    'cunit2': 'arcsec',
```

```
'cunit3': 'angstrom',    'ctype1': 'hpln-tan',    'ctype2': 'hplt-tan',    'ctype3':
```

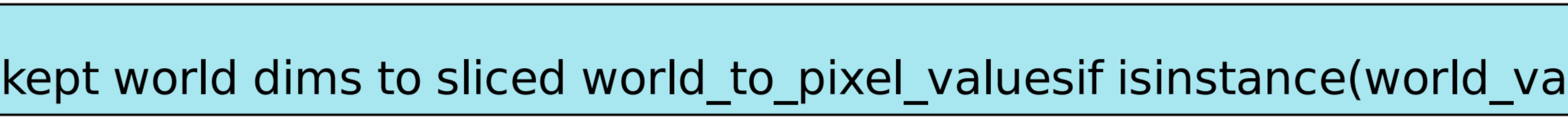
```
'wave',    'crval1': 0.0,    'crval2': 0.0,    'crval3': 1.05,}w = wcs(header=wcs_h
```

```
eader)# world values at pz=0 (the sliced plane)px, py, pz = 49.5, 12.,
```



```
0.0world_vals0 = w.pixel_to_world_values(px, py, pz)full_px0 = w.world_to_p
```

```
ixel_values(*world_vals0)ll = slicedlowlevelwcs(w, 0)# pass only the
```

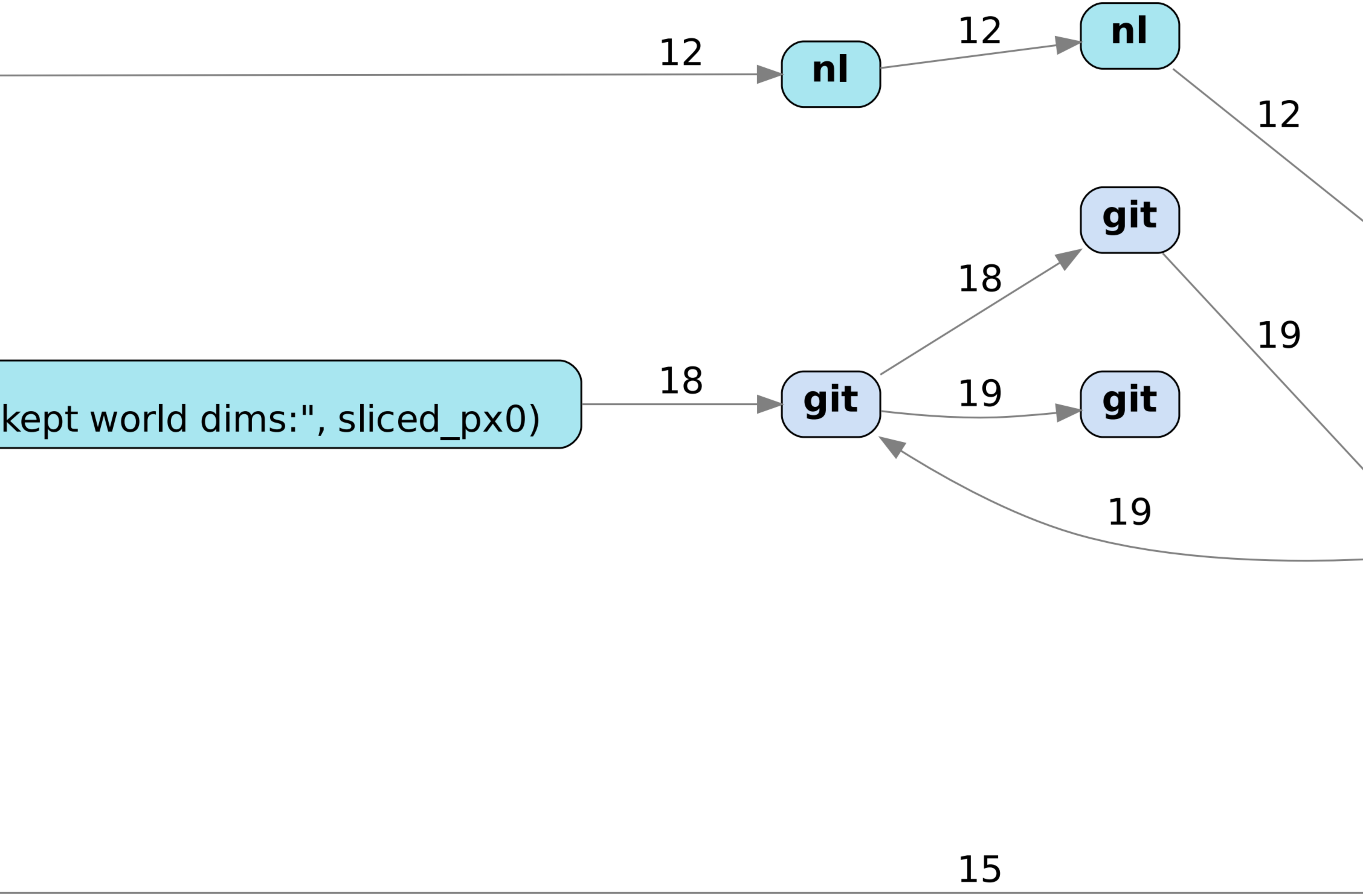

kept world dims to sliced world_to_pixel_valuesif isinstance(world_va

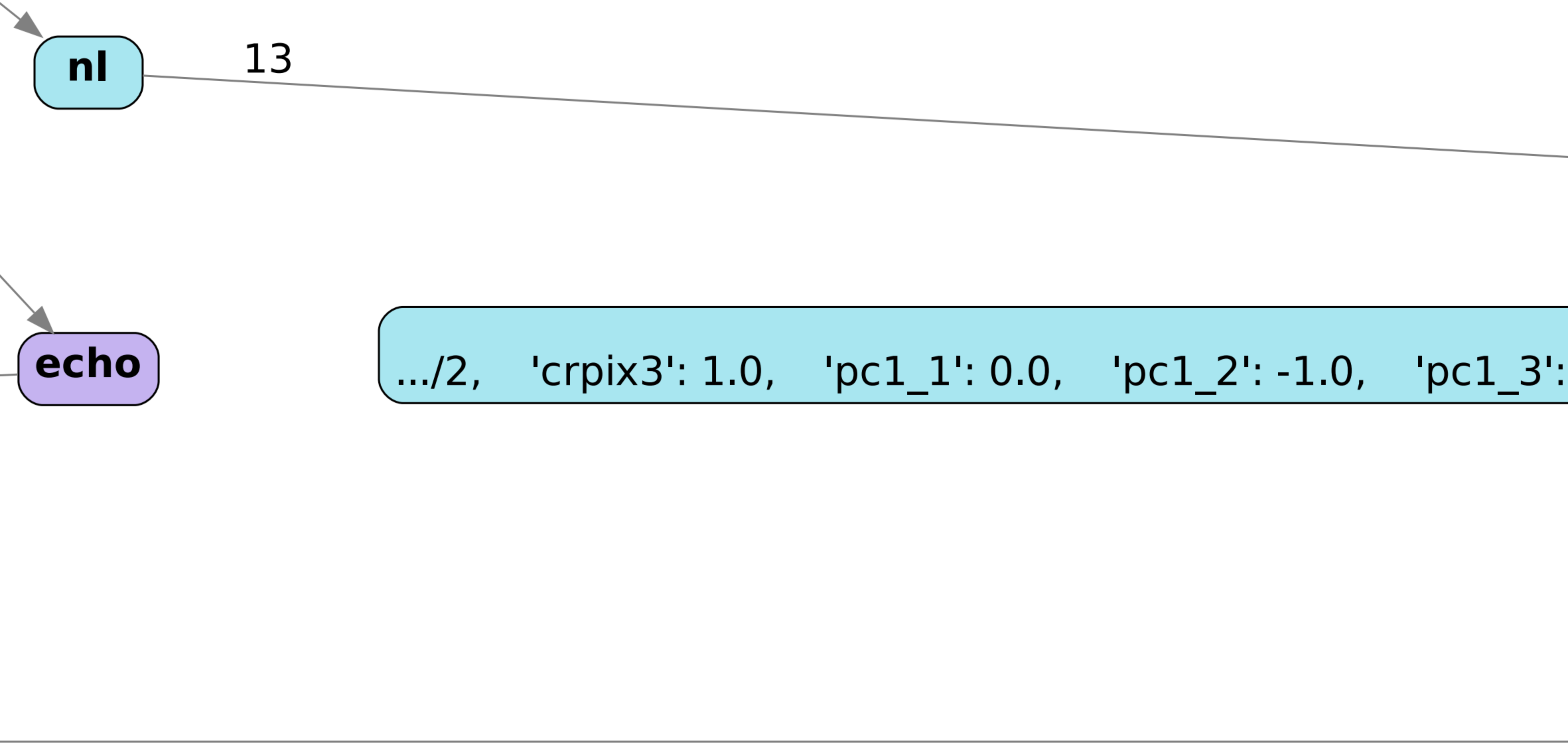
```
ls0, tuple):    args = list(world_vals0)[:ll.world_n_dim]else:    args =
```

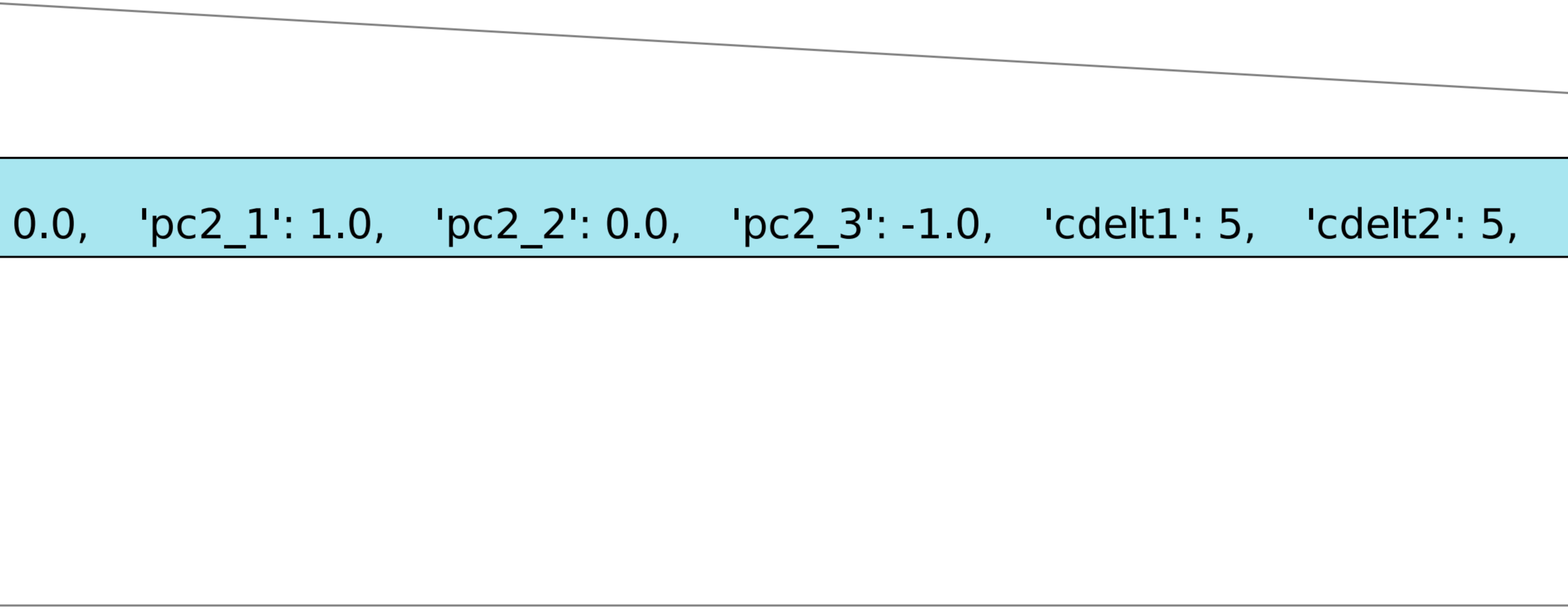
```
[world_vals0]sliced_px0 = ll.world_to_pixel_values(*args)print("world_va
```

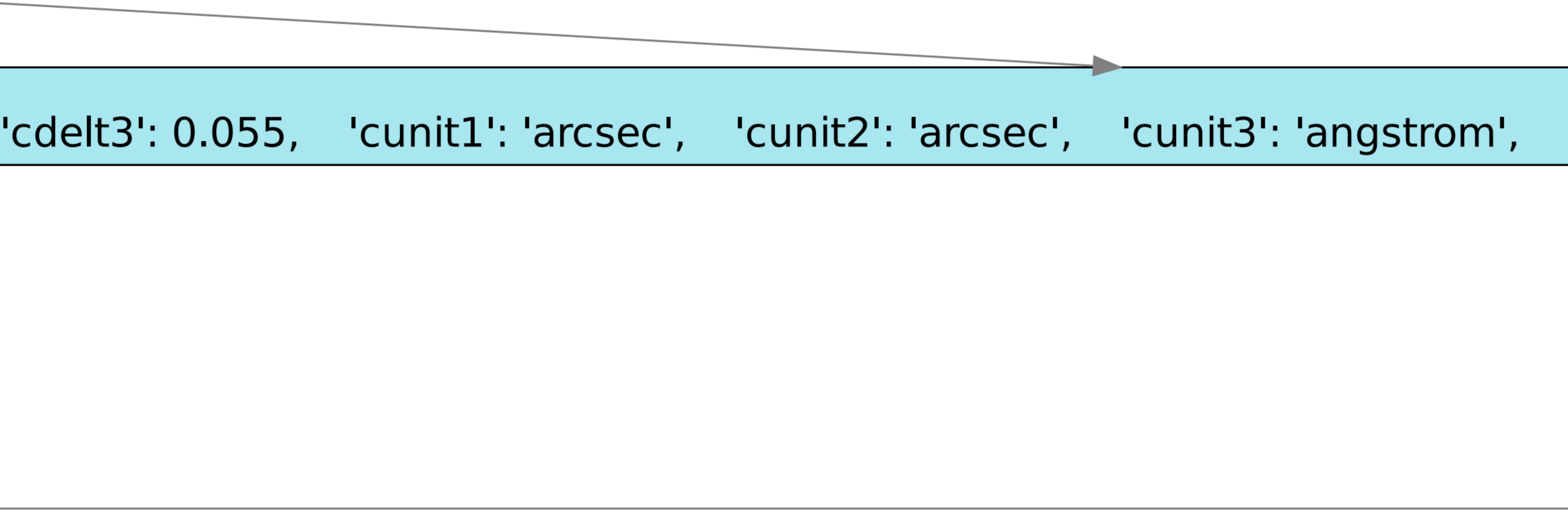
```
ls0 (at pz=0):", world_vals0)print("world_to_pixel_values (full) for
```

```
those world_vals0:", full_px0)print("world_to_pixel_values (sliced) for
```







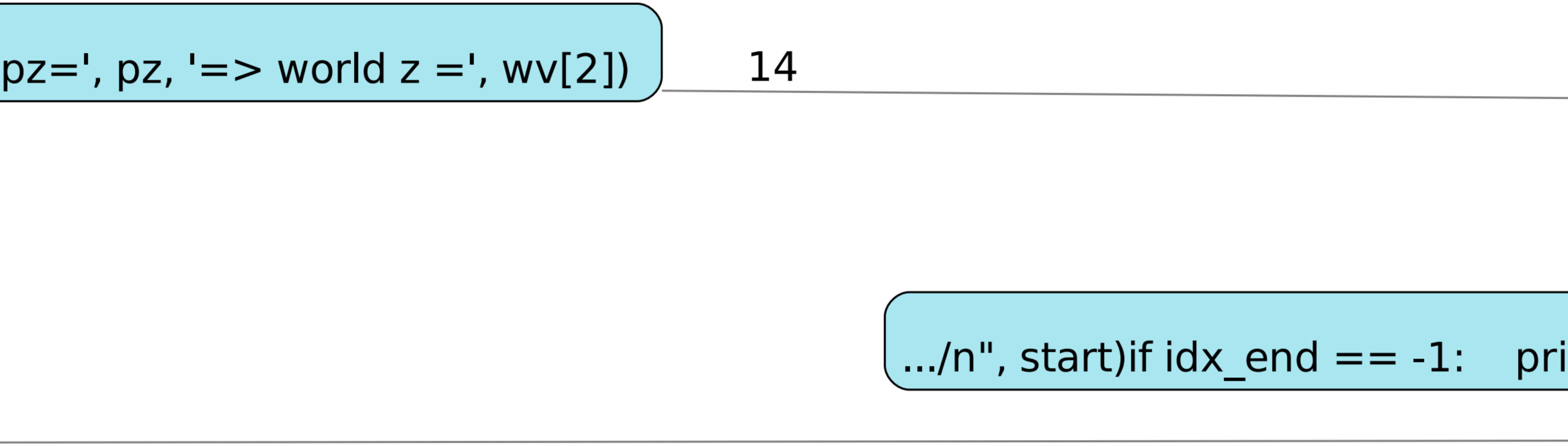


python

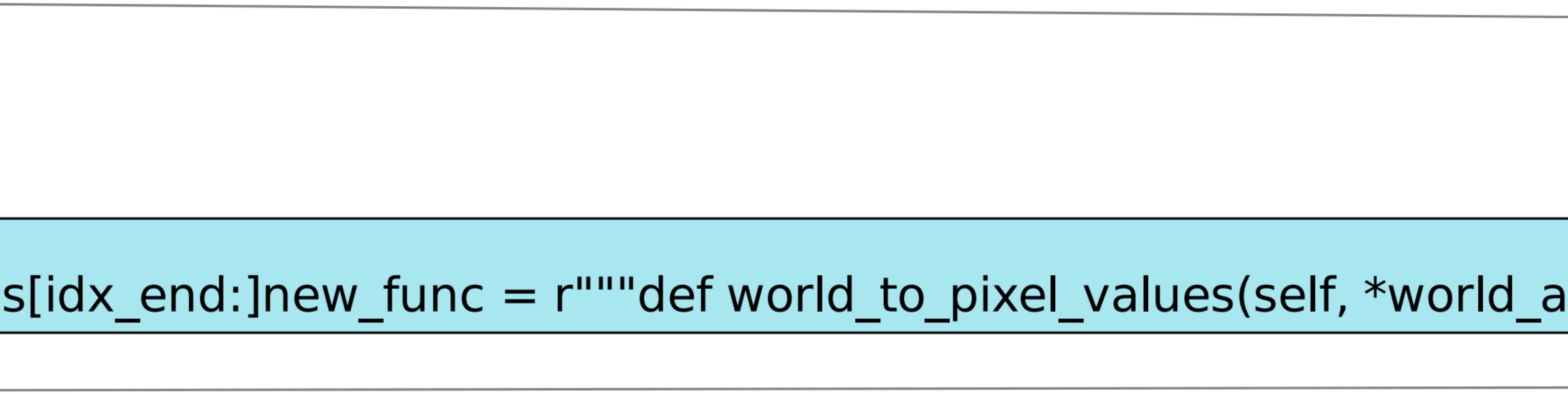
```
'ctype1': 'hpln-tan',    'ctype2': 'hplt-tan',    'ctype3': 'wave',    'crval1': 0.0,    'c
```

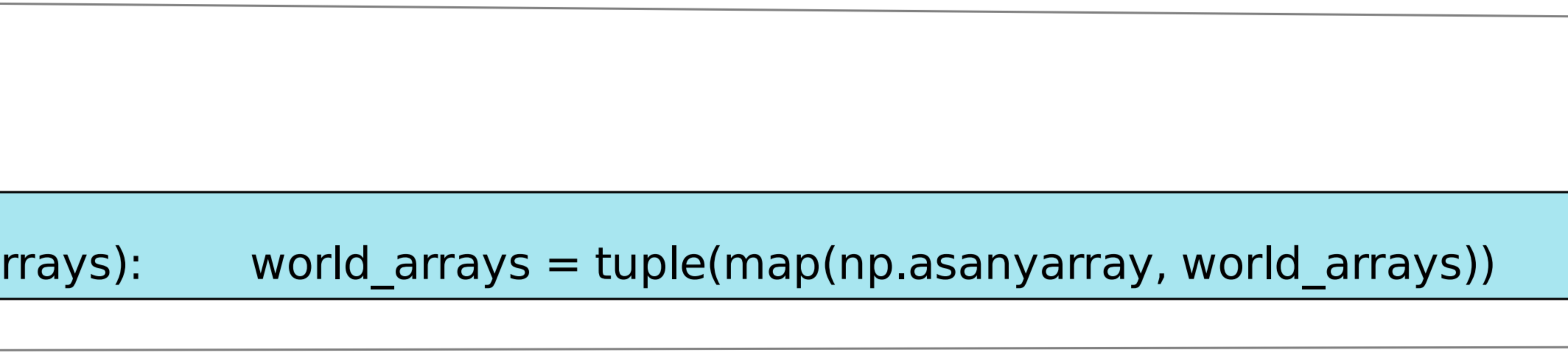
```
rval2': 0.0,    'crval3': 1.05,}w=wcs(header=wcs_header)px,py = 49.5, 12.for
```

```
pz in (0.0, 1.0):    wv = w.pixel_to_world_values(px, py, pz)    print('
```


pz=', pz, '=> world z =', wv[2])
14
.../n", start)if idx_end == -1:   pri

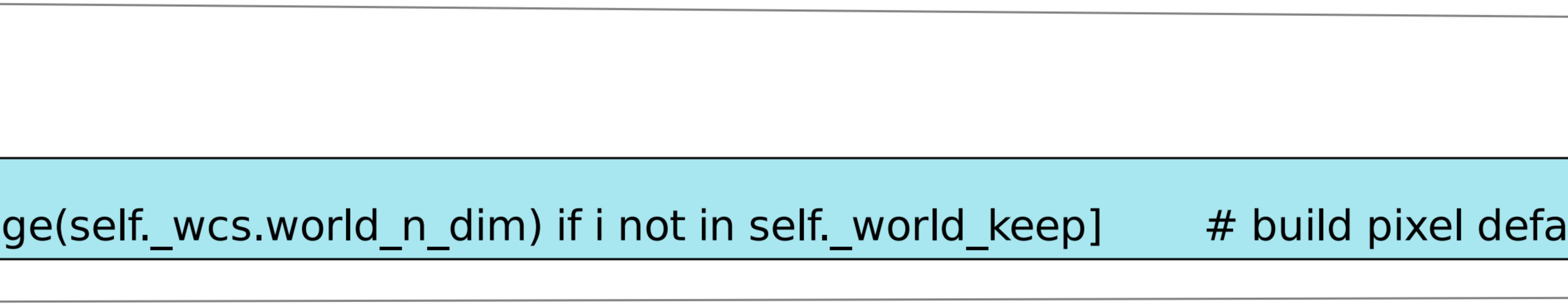
```
nt("end marker not found"); raise systemexit(1)head = s[:start]tail =
```


s[idx_end:]new_func = r"""def world_to_pixel_values(self, *world_a



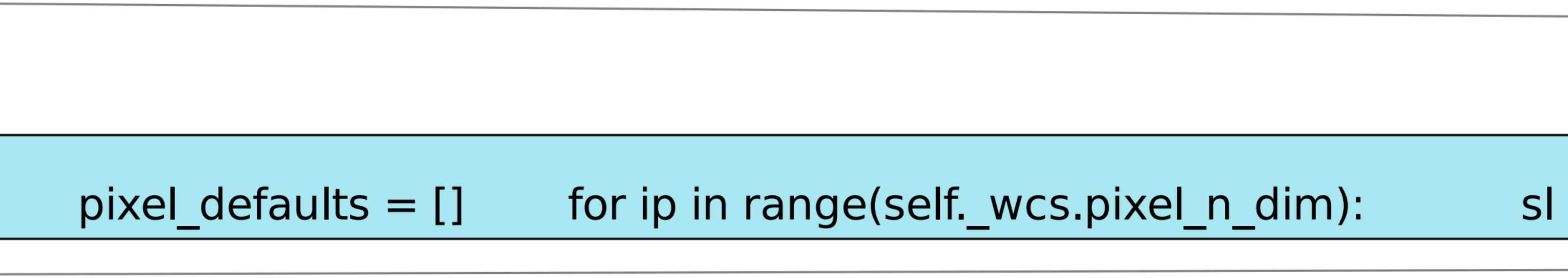
```
rrays):       world_arrays = tuple(map(np.asanyarray, world_arrays))
```

```
# determine dropped world indices      dropped_indices = [i for i in ran
```

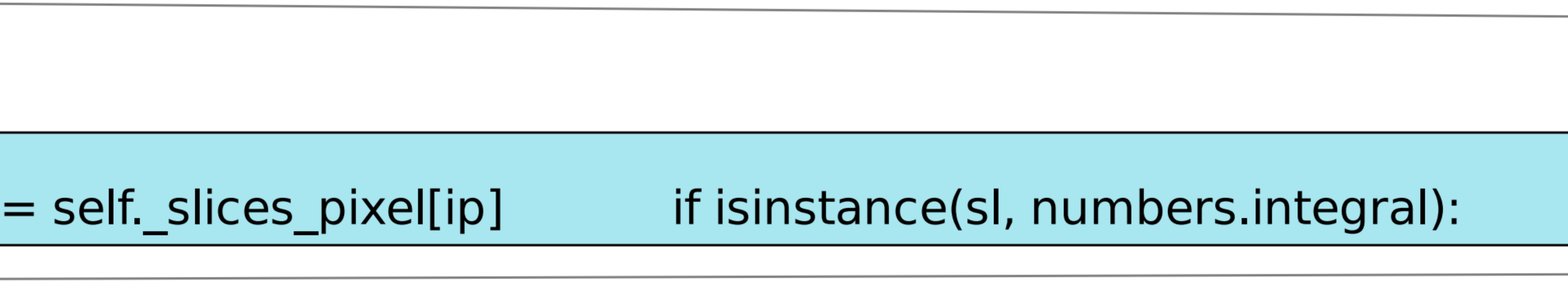


```
ge(self._wcs.world_n_dim) if i not in self._world_keep]        # build pixel defa
```

ults from slices: use integer indices when present, else slice start if set, else 0.

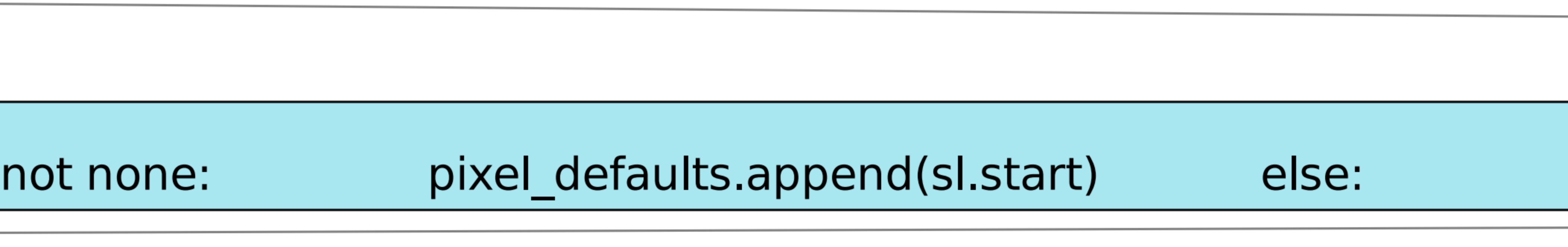


```
pixel_defaults = []     for ip in range(self._wcs.pixel_n_dim):     sl
```



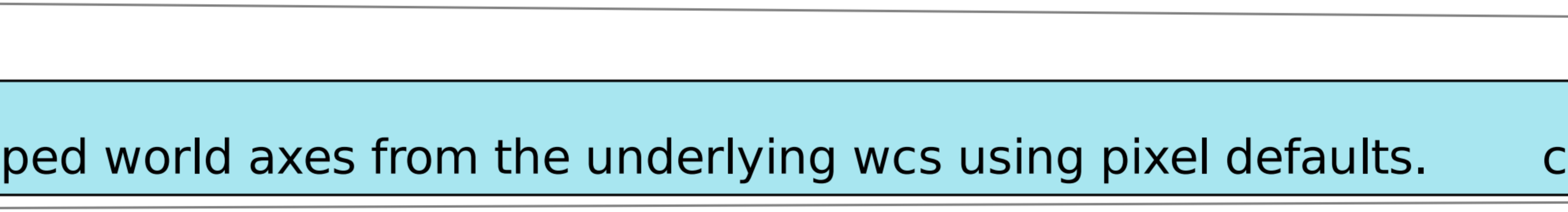
```
= self._slices_pixel[ip]        if isinstance(sl, numbers.integral):
```

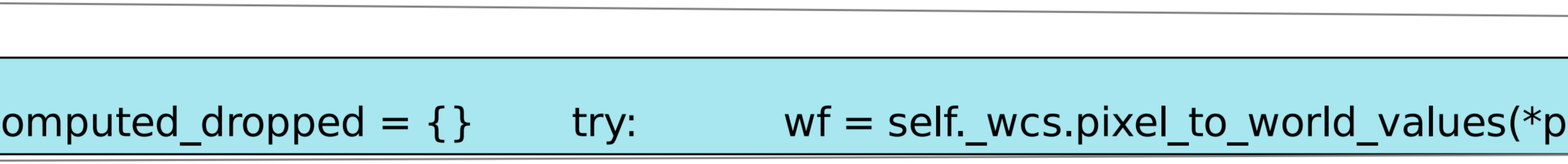
```
pixel_defaults.append(sl)        elif isinstance(sl, slice) and sl.start is
```



```
not none:          pixel_defaults.append(sl.start)        else:
```

```
pixel_defaults.append(0)      # try to compute world values for drop
```

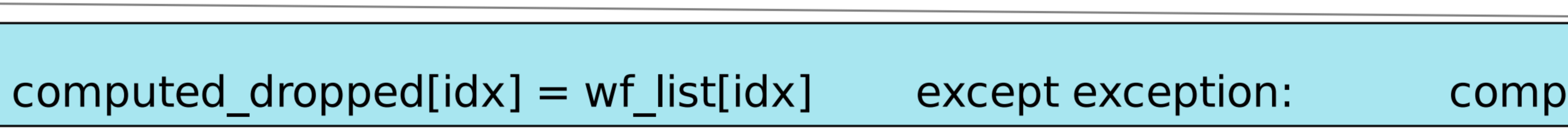

ped world axes from the underlying wcs using pixel defaults.
c



```
omputed_dropped = {}       try:           wf = self._wcs.pixel_to_world_values(*p
```

```
ixel_defaults)          if isinstance(wf, tuple):          wf_list = list(wf)
```

```
else:            wf_list = [wf]         for idx in dropped_indices:
```



```
computed_dropped[idx] = wf_list[idx]      except exception:        comp
```

```
uted_dropped = {}       # build the full list of world arrays, inserting provided
```

```
ones for kept axes and      # either computed dropped values or fallbacks for
```

```
dropped axes.        world_arrays_new = []        iworld_curr = -1        try:        d
```

```
ropped_info = self.dropped_world_dimensions        dropped_values = drop
```

python

```
ped_info.get('value', []) if isinstance(dropped_info, dict) else []        except
```

```
exception:          dropped_values = []        for iworld in range(self._wcs
```

```
.world_n_dim):        if iworld in self._world_keep:            iworld_curr += 1
```

```
world_arrays_new.append(world_arrays[iworld_curr])        else:
```

```
if iworld in computed_dropped:                world_arrays_new.append(com
```

```
puted_dropped[iworld])          elif dropped_values:          # map drop
```

ped_values list order to dropped indices order

di = dropped_indices

```
.index(iworld)               world_arrays_new.append(dropped_values[di])
```

```
else:            world_arrays_new.append(1.)     # broadcast to matching
```

shapes and compute pixel positions world_arrays_new = np.broadcast_a

rrays(*world_arrays_new)
pixel_arrays = list(self._wcs.world_to_pixel_val

```
ues(*world_arrays_new))        # subtract slice starts for non-integer sliced p
```

```
ixels        for ipixel in range(self._wcs.pixel_n_dim):          if isinstance(self._sl
```

```
ices_pixel[ipixel], slice) and self._slices_pixel[ipixel].start is not none:
```

```
pixel_arrays[ipixel] -= self._slices_pixel[ipixel].start        # detect the case of a
```

length 0 array if isinstance(pixel_arrays, np.ndarray) and not pixel_arrays

```
.shape:        return pixel_arrays      # return only the kept pixel dimensions
```

```
pixel = tuple(pixel_arrays[ip] for ip in self._pixel_keep)        if self.pixel_n_dim
```

```
== 1 and self._wcs.pixel_n_dim > 1:          pixel = pixel[0]        return pixel"""#
```

assemble and writes2 = head + new_func + tailp.write_text(s2)print("patched")